LAPORAN PRAKTIKUM

PEMROGRAMAN MOBILE

Pertemuan 3: Form Input dan State Management

Nama	:	Muhamad Dika Ramadhan
NIM	:	20240801045
Program Studi	:	Teknik Informatika
Mata Kuliah	:	Pemrograman Mobile
Pertemuan	:	3

1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Mengambil input pengguna dengan TextField dan TextEditingController.
2. Membuat form dengan validasi menggunakan Form, TextFormField, dropdown, dan checkbox.
3. Menjelaskan keterbatasan setState ketika data dipakai oleh banyak halaman.
4. Menerapkan state management dasar dengan ChangeNotifier dan paket Provider.
5. Membedakan penggunaan context.watch dan context.read.

2. Landasan Teori

Input dan Form

TextField menerima input teks, dan nilainya dibaca lewat TextEditingController. Untuk beberapa isian yang perlu divalidasi sekaligus digunakan Form dengan GlobalKey<FormState>, lalu method validate() dipanggil. Setiap TextFormField memiliki properti validator yang mengembalikan pesan error bila isian tidak valid, atau null bila valid.

State dan State Management

State adalah data yang dapat berubah dan memengaruhi tampilan. Berdasarkan cakupannya, state dibedakan sebagai berikut:

Jenis State	Cakupan	Contoh Alat
Ephemeral (lokal)	Satu widget	setState, checkbox tercentang

Jenis State	Cakupan	Contoh Alat
App state (global)	Banyak widget/halaman	Provider, Riverpod, Bloc; daftar tugas, keranjang belanja

Provider menempatkan objek state (turunan ChangeNotifier) di atas pohon widget. Widget yang membutuhkannya cukup "berlangganan"; ketika model memanggil notifyListeners(), widget tersebut dibangun ulang secara otomatis.

Perintah	Fungsi	Dipakai di
context.watch<T>()	Membaca data dan ikut membangun ulang widget saat data berubah.	Di dalam build()
context.read<T>()	Membaca sekali tanpa membangun ulang widget.	Di dalam callback (onPressed, dll.)

3. Alat dan Bahan

- Flutter SDK, editor, dan emulator dari pertemuan sebelumnya
- Proyek baru: flutter create praktikum_3
- Koneksi internet untuk mengunduh paket provider

4. Langkah dan Hasil Praktikum

Halaman Menu Utama

Agar ketiga bagian praktikum dapat dibuka dari satu aplikasi, dibuat halaman menu berisi tiga tombol yang berpindah ke halaman masing-masing dengan Navigator.push.

```
class MenuUtama extends StatelessWidget {
  const MenuUtama({super.key});

  Widget _tombol(BuildContext context, String label,
      Widget halaman) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 12),
      child: ElevatedButton(
        onPressed: () => Navigator.push(context,
            MaterialPageRoute(builder: (_) => halaman)),
        child: Text(label),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Praktikum 3 - Menu')),
```

```
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          _tombol(context, 'Bagian A: Input Dasar (TextField)',
              const InputPage()),
          _tombol(context, 'Bagian B: Form Validasi',
              const FormPage()),
          _tombol(context, 'Bagian D: Aplikasi Tugas (Provider)',
              const TugasPage()),
        ],
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
MenuUtama (StatelessWidget)	Halaman awal berisi daftar tombol navigasi; tidak punya data yang berubah.
_tombol(context, label, halaman)	Method pembantu yang membuat satu tombol. Dengan method ini kode tidak perlu diulang tiga kali.
Navigator.push + MaterialPageRoute	Membuka halaman tujuan di atas stack halaman.
ListView(padding: ...)	Induk berupa daftar yang dapat di-scroll sehingga aman dari overflow.

Bagian A. Input Dasar dengan TextField

```
class _InputPageState extends State<InputPage> {
  final _controller = TextEditingController();
  String _salam = '';

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Input Dasar')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              decoration: const InputDecoration(
                labelText: 'Nama',
```

```
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: () {
                setState(() =>
                    _salam = 'Halo, ${_controller.text}!');
              },
              child: const Text('Sapa'),
            ),
            const SizedBox(height: 12),
            Text(_salam,
                style: const TextStyle(fontSize: 20)),
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
InputPage extends StatefulWidget	Halaman bersifat stateful karena teks sapaan berubah setelah tombol ditekan.
TextEditingController()	Objek pengendali TextField untuk membaca atau mengubah teks yang diketik.
String _salam = ''	Variabel state penampung teks sapaan, awalnya kosong.
dispose()	Dipanggil saat halaman ditutup. _controller.dispose() melepas controller agar tidak terjadi kebocoran memori.
super.dispose()	Memanggil dispose milik kelas induk agar proses pembersihan bawaan Flutter tetap berjalan.
TextField(controller: _controller)	Kolom input teks yang dihubungkan dengan controller.
InputDecoration(labelText, border)	Mengatur tampilan kolom: label "Nama" dan garis tepi OutlineInputBorder.
_controller.text	Mengambil teks yang saat ini ada di kolom input.
setState(() => _salam = ...)	Mengisi _salam lalu memicu build() ulang agar sapaan tampil.
'Halo, ${...}!'	String interpolation untuk menyisipkan nama ke dalam teks sapaan.

Setelah nama diketik lalu tombol Sapa ditekan, teks "Halo, dika!" tampil di bawah tombol (Gambar 2).

Bagian B. Form dengan Validasi

```
class _FormPageState extends State<FormPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _email = TextEditingController();
  String? _jurusan;
  bool _setuju = false;

  @override
  void dispose() {
    _nama.dispose();
    _email.dispose();
    super.dispose();
  }

  void _kirim() {
    if (_formKey.currentState!.validate()) {
      final jurusan = _jurusan ?? '-';
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(
            'Terdaftar: ${_nama.text} ($jurusan)')),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Form Pendaftaran')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(
                labelText: 'Nama lengkap',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                  (v == null || v.trim().isEmpty)
                      ? 'Nama wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _email,
              keyboardType: TextInputType.emailAddress,
              decoration: const InputDecoration(
                labelText: 'Email',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
```

```
                if (v == null || !v.contains('@')) {
                  return 'Email tidak valid';
                }
                return null;
              },
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              decoration: const InputDecoration(
                labelText: 'Jurusan',
                border: OutlineInputBorder(),
              ),
              items: const [
                DropdownMenuItem(value: 'TI',
                    child: Text('Teknik Informatika')),
                DropdownMenuItem(value: 'SI',
                    child: Text('Sistem Informasi')),
                DropdownMenuItem(value: 'TE',
                    child: Text('Teknik Elektro')),
              ],
              onChanged: (v) => setState(() => _jurusan = v),
              validator: (v) => v == null ? 'Pilih jurusan' : null,
            ),
            CheckboxListTile(
              title: const Text('Saya menyetujui ketentuan'),
              value: _setuju,
              controlAffinity: ListTileControlAffinity.leading,
              onChanged: (v) =>
                  setState(() => _setuju = v ?? false),
            ),
            ElevatedButton(
              onPressed: _setuju ? _kirim : null,
              child: const Text('Daftar'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
GlobalKey<FormState>	Kunci unik untuk mengakses state Form sehingga validate() dapat dipanggil dari luar Form.
Form(key: _formKey)	Pembungkus yang mengelompokkan seluruh isian agar divalidasi bersamaan.
TextFormField	Versi TextField yang terintegrasi dengan Form dan memiliki properti validator.

Bagian Kode	Penjelasan
validator: (v) => ...	Fungsi pemeriksa isian. Mengembalikan pesan error (muncul merah di bawah kolom) atau null jika valid.
v.trim().isEmpty	Menghapus spasi di awal/akhir lalu memeriksa apakah kosong.
v.contains('@')	Memeriksa sederhana apakah email memuat tanda @.
keyboardType: TextInputType.emailAddress	Menampilkan keyboard yang sesuai untuk mengetik email.
String? _jurusan	Tipe nullable: bernilai null sampai pengguna memilih jurusan.
DropdownButtonFormField<String>	Menu pilihan (dropdown) yang ikut divalidasi oleh Form.
DropdownMenuItem(value, child)	Satu pilihan dalam dropdown; value adalah nilai yang disimpan, child adalah teks yang tampil.
onChanged	Callback saat pilihan berubah; menyimpan nilai ke _jurusan lewat setState.
CheckboxListTile	Baris berisi checkbox dan judul. controlAffinity.leading menaruh checkbox di kiri.
bool _setuju	State yang menyimpan status centang.
onPressed: _setuju ? _kirim : null	Jika onPressed bernilai null, tombol otomatis nonaktif; jadi tombol Daftar baru aktif setelah checkbox dicentang.
_formKey.currentState!.validate()	Menjalankan semua validator; bernilai true bila seluruh isian valid.
_jurusan ?? '-'	Operator null-coalescing: memakai "-" bila _jurusan masih null.
ScaffoldMessenger...showSnackBar	Menampilkan pesan singkat (SnackBar) di bagian bawah layar.

Bagian C. Mengapa Butuh State Management?

Pada Bagian A dan B, state hidup di dalam satu halaman dan dikelola dengan setState. Pada aplikasi daftar tugas, halaman daftar dan halaman tambah perlu berbagi data yang sama. Mengoper data lewat constructor dan callback dari satu halaman ke halaman lain cepat menjadi rumit seiring bertambahnya halaman. Solusinya adalah memindahkan data ke satu objek bersama yang dapat diakses kedua halaman, dan inilah peran Provider.

Bagian D. Aplikasi Daftar Tugas dengan Provider

Langkah 1. Memasang paket provider di folder proyek:

```
flutter pub add provider
```

Langkah 2. Membuat model dan entry point:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;
  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
      _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ),
  );
}
```

Bagian Kode	Penjelasan
import 'package:provider/provider.dart'	Mengimpor paket Provider.

Bagian Kode	Penjelasan
class Tugas	Model data satu tugas: judul dan status selesai. {this.selesai = false} adalah named parameter opsional dengan nilai awal false.
class TugasModel extends ChangeNotifier	Model state bersama. ChangeNotifier menyediakan kemampuan memberi tahu pendengar saat data berubah.
final List<Tugas> _items	Daftar tugas bersifat private (diawali _) agar hanya bisa diubah lewat method model.
get items => List.unmodifiable(_items)	Getter yang memberi salinan daftar yang tidak bisa diubah dari luar, sehingga data terlindungi.
get jumlahSelesai	Getter terhitung; where() menyaring tugas yang selesai lalu length menghitung jumlahnya.
tambah(judul)	Menambahkan tugas baru lalu memanggil notifyListeners().
toggle(index)	Membalik status selesai tugas pada index tertentu.
hapus(index)	Menghapus tugas pada index tertentu dengan removeAt().
notifyListeners()	Memberi tahu semua widget yang berlangganan agar dibangun ulang. Tanpa ini tampilan tidak berubah.
ChangeNotifierProvider(create: ...)	Menyediakan satu instance TugasModel ke seluruh pohon widget di bawahnya. Ditaruh di atas MyApp agar semua halaman dapat mengaksesnya.

Langkah 3. Membuat halaman daftar tugas:

```
class TugasPage extends StatelessWidget {
  const TugasPage({super.key});

  @override
  Widget build(BuildContext context) {
    final model = context.watch<TugasModel>();
    return Scaffold(
      appBar: AppBar(
        title: Text('Tugas (${model.jumlahSelesai}/'
            '${model.items.length})'),
      ),
      body: ListView.builder(
        itemCount: model.items.length,
        itemBuilder: (context, i) {
          final t = model.items[i];
          return ListTile(
            leading: Checkbox(
              value: t.selesai,
              onChanged: (_) =>
                  context.read<TugasModel>().toggle(i),
            ),
```

```
            title: Text(
              t.judul,
              style: TextStyle(
                decoration: t.selesai
                    ? TextDecoration.lineThrough : null,
              ),
            ),
            trailing: IconButton(
              icon: const Icon(Icons.delete),
              onPressed: () =>
                  context.read<TugasModel>().hapus(i),
            ),
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          Navigator.push(context, MaterialPageRoute(
              builder: (_) => const TambahPage()));
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
context.watch<TugasModel>()	Mengambil model sekaligus berlangganan; TugasPage dibangun ulang setiap kali model berubah.
title: Text('Tugas (selesai/total)')	Judul AppBar menampilkan jumlah tugas selesai dibanding total, contoh 1/3.
ListView.builder	Membangun daftar sesuai jumlah tugas di model.
Checkbox(value, onChanged)	Kotak centang penanda tugas selesai; saat diubah memanggil toggle(i).
context.read<TugasModel>()	Mengambil model tanpa berlangganan; dipakai di dalam callback karena hanya perlu memanggil method.
TextDecoration.lineThrough	Mencoret judul tugas yang sudah selesai.
IconButton(Icons.delete)	Tombol ikon sampah yang memanggil hapus(i).
FloatingActionButton	Tombol + untuk berpindah ke halaman TambahPage.

Langkah 4. Membuat halaman tambah tugas yang memakai model yang sama:

```
class _TambahPageState extends State<TambahPage> {
  final _controller = TextEditingController();

  @override
```

```
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

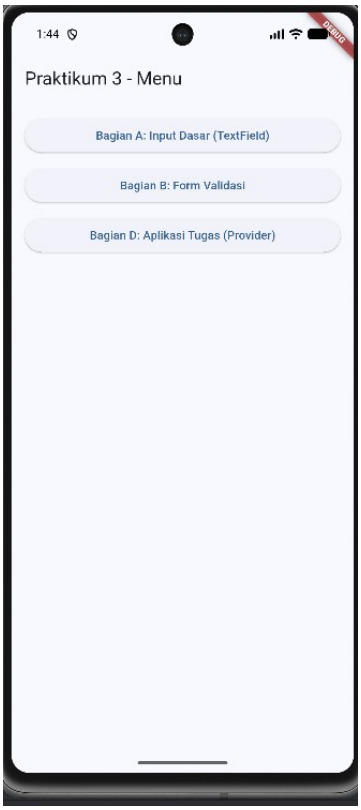
  void _simpan() {
    final judul = _controller.text.trim();
    if (judul.isEmpty) return;
    context.read<TugasModel>().tambah(judul);
    Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tambah Tugas')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _controller,
              autofocus: true,
              decoration: const InputDecoration(
                labelText: 'Judul tugas',
                border: OutlineInputBorder(),
              ),
              onSubmitted: (_) => _simpan(),
            ),
            const SizedBox(height: 12),
            ElevatedButton(onPressed: _simpan,
                child: const Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

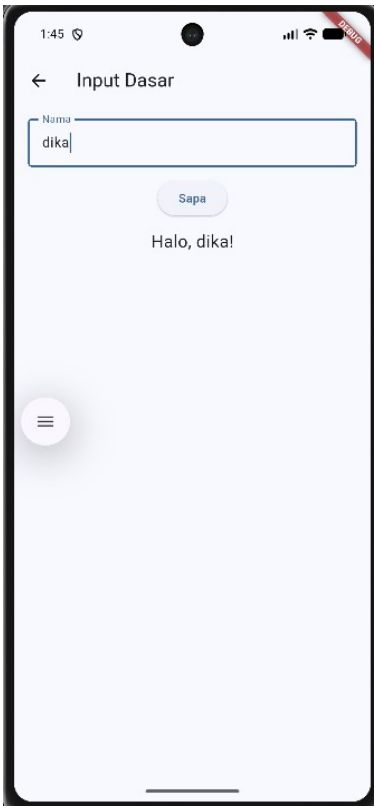
Bagian Kode	Penjelasan
_simpan()	Mengambil teks, membatalkan bila kosong, menambahkan ke model, lalu menutup halaman.
_controller.text.trim()	Mengambil teks tanpa spasi di awal/akhir.
context.read<TugasModel>().tambah(judul)	Memanggil model yang sama dengan halaman daftar, sehingga tugas langsung muncul di sana tanpa mengoper data lewat constructor.
Navigator.pop(context)	Kembali ke halaman daftar.

Bagian Kode	Penjelasan
autofocus: true	Kolom input langsung aktif dan keyboard terbuka saat halaman dibuka.
onSubmitted	Menjalankan _simpan() saat tombol Enter pada keyboard ditekan.

Hasil Praktikum



Gambar 1. Halaman menu utama



Gambar 2. Input Dasar: sapaan setelah tombol Sapa ditekan (Bagian A)

Gambar 1 menampilkan menu utama dengan tiga tombol ke Bagian A, Bagian B, dan Bagian D. Gambar 2 menampilkan hasil Bagian A: nama "dika" yang diketik muncul sebagai "Halo, dika!". Hasil Bagian D (aplikasi tugas dengan Provider) ditampilkan pada bagian Latihan Mandiri di bawah.

5. Latihan Mandiri

Latihan mandiri dikerjakan pada aplikasi Daftar Tugas: (1) validasi judul minimal 3 karakter memakai Form dan TextFormField, (2) method hapusSelesai() beserta tombol di AppBar, (3) SnackBar "Tugas ditambahkan", dan (4) teks "Belum ada tugas" saat daftar kosong.

Method hapusSelesai() pada model

```
// di TugasModel
void hapusSelesai() {
  _items.removeWhere((t) => t.selesai);
  notifyListeners();
}
```

Tombol di AppBar dan tampilan daftar kosong

```
// di TugasPage: AppBar dengan tombol hapus selesai
appBar: AppBar(
  title: Text('Tugas (${model.jumlahSelesai}/'
      '${model.items.length})'),
  actions: [
    IconButton(
      icon: const Icon(Icons.delete_sweep),
      onPressed: model.jumlahSelesai > 0
          ? () => context.read<TugasModel>().hapusSelesai()
          : null,   // nonaktif bila belum ada yang selesai
    ),
  ],
),

// di TugasPage: tampilan saat daftar kosong
body: model.items.isEmpty
    ? const Center(
        child: Text('Belum ada tugas',
            style: TextStyle(color: Colors.grey)))
    : ListView.builder( /* ... seperti sebelumnya ... */ ),
```

TambahPage dengan validasi dan SnackBar

```
class _TambahPageState extends State<TambahPage> {
  final _formKey = GlobalKey<FormState>();
  final _controller = TextEditingController();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void _simpan() {
    if (!_formKey.currentState!.validate()) return;
    context.read<TugasModel>().tambah(_controller.text.trim());
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Tugas ditambahkan')),
    );
    Navigator.pop(context);
  }

  @override
```

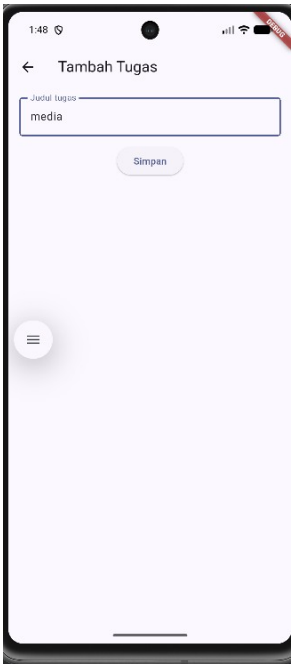
```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tambah Tugas')),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: _controller,
                autofocus: true,
                decoration: const InputDecoration(
                  labelText: 'Judul tugas',
                  border: OutlineInputBorder(),
                ),
                validator: (v) {
                  if (v == null || v.trim().length < 3) {
                    return 'Judul minimal 3 karakter';
                  }
                  return null;
                },
                onFieldSubmitted: (_) => _simpan(),
              ),
              const SizedBox(height: 12),
              ElevatedButton(onPressed: _simpan,
                  child: const Text('Simpan')),
            ],
          ),
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
hapusSelesai()	Menghapus semua tugas selesai sekaligus dengan removeWhere(), lalu notifyListeners() agar daftar diperbarui.
actions: [IconButton(...)]	Daftar tombol di sisi kanan AppBar; di sini tombol ikon delete_sweep.
onPressed: jumlahSelesai > 0 ? ... : null	Tombol hanya aktif bila ada tugas selesai. Bila null, tombol tampil abu-abu dan tidak bisa ditekan (terlihat pada Gambar 3).
model.items.isEmpty ? Center(...) : ListView...	Operator ternary: tampilkan teks "Belum ada tugas" bila kosong, selain itu tampilkan daftar.
Form + TextFormField	Mengganti TextField agar bisa divalidasi.

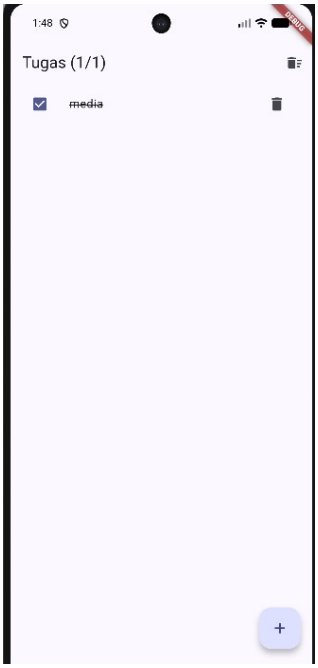
Bagian Kode	Penjelasan
v.trim().length < 3	Syarat validasi: judul setelah dipotong spasi minimal 3 karakter, jika tidak muncul pesan error.
validate()	Dipanggil di _simpan(); bila tidak valid proses simpan dihentikan dengan return.
showSnackBar(SnackBar(...))	Menampilkan notifikasi "Tugas ditambahkan" setelah tugas tersimpan.
onFieldSubmitted	Padanan onSubmitted untuk TextFormField, memicu _simpan() saat Enter ditekan.



Gambar 3. Daftar kosong: teks "Belum ada tugas"



Gambar 4. Halaman Tambah Tugas



Gambar 5. Tugas dicentang dan dicoret, judul AppBar menjadi 1/1

Gambar 3 menampilkan daftar kosong dengan teks "Belum ada tugas" dan tombol hapus selesai yang masih nonaktif (abu-abu). Gambar 4 adalah halaman kedua untuk menambah tugas "media" yang berbagi model yang sama. Pada Gambar 5 tugas tersebut muncul di daftar, dicentang sehingga teksnya tercoret, judul AppBar berubah menjadi "Tugas (1/1)", dan tombol hapus selesai menjadi aktif.

6. Tugas: Aplikasi Daftar Belanja

Aplikasi Daftar Belanja memiliki halaman form tambah (nama barang wajib, jumlah wajib berupa angka lebih dari 0, dan kategori lewat dropdown, semuanya divalidasi) serta halaman daftar yang dapat mencentang barang sebagai "sudah dibeli" dan menghapusnya. State disimpan pada satu

ChangeNotifier dan dibagikan dengan Provider; AppBar menampilkan jumlah barang yang belum dibeli.

Model state dan entry point

```
class Barang {
  String nama;
  int jumlah;
  String kategori;
  bool sudahDibeli;
  Barang(this.nama, this.jumlah, this.kategori,
      {this.sudahDibeli = false});
}

class BelanjaModel extends ChangeNotifier {
  final List<Barang> _items = [];

  List<Barang> get items => List.unmodifiable(_items);

  int get jumlahBelumDibeli =>
      _items.where((b) => !b.sudahDibeli).length;

  void tambah(String nama, int jumlah, String kategori) {
    _items.add(Barang(nama, jumlah, kategori));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].sudahDibeli = !_items[index].sudahDibeli;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => BelanjaModel(),
      child: const MyApp(),   // MaterialApp: tema teal
    ),
  );
}
```

Halaman daftar belanja

```
class BelanjaPage extends StatelessWidget {
  const BelanjaPage({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    final model = context.watch<BelanjaModel>();
    return Scaffold(
      appBar: AppBar(
        title: Text('Daftar Belanja '
            '(${model.jumlahBelumDibeli} Belum Dibeli)'),
      ),
      body: model.items.isEmpty
          ? const Center(
              child: Text('Daftar belanja masih kosong',
                  style: TextStyle(color: Colors.grey)))
          : ListView.builder(
              itemCount: model.items.length,
              itemBuilder: (context, i) {
                final b = model.items[i];
                return Card(
                  margin: const EdgeInsets.symmetric(
                      horizontal: 12, vertical: 6),
                  child: ListTile(
                    leading: Checkbox(
                      value: b.sudahDibeli,
                      onChanged: (_) => context
                          .read<BelanjaModel>().toggle(i),
                    ),
                    title: Text(b.nama,
                        style: TextStyle(
                          fontWeight: FontWeight.bold,
                          decoration: b.sudahDibeli
                              ? TextDecoration.lineThrough
                              : null,
                        )),
                    subtitle: Text('Jumlah: ${b.jumlah} | '
                        'Kategori: ${b.kategori}'),
                    trailing: IconButton(
                      icon: const Icon(Icons.delete,
                          color: Colors.redAccent),
                      onPressed: () => context
                          .read<BelanjaModel>().hapus(i),
                    ),
                  ),
                );
              },
            ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => Navigator.push(context,
            MaterialPageRoute(
                builder: (_) => const TambahBarangPage())),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Halaman form tambah barang

```
class _TambahBarangPageState extends State<TambahBarangPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _jumlah = TextEditingController();
  String? _kategori;

  static const _daftarKategori = [
    'Makanan', 'Minuman', 'Kebutuhan Rumah', 'Lainnya'
  ];

  @override
  void dispose() {
    _nama.dispose();
    _jumlah.dispose();
    super.dispose();
  }

  void _simpan() {
    if (_formKey.currentState!.validate()) {
      context.read<BelanjaModel>().tambah(
        _nama.text.trim(),
        int.parse(_jumlah.text),
        _kategori!,
      );
      Navigator.pop(context);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
          title: const Text('Tambah Barang Belanjaan')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _nama,
              decoration: const InputDecoration(
                labelText: 'Nama Barang',
                border: OutlineInputBorder(),
              ),
              validator: (v) =>
                  (v == null || v.trim().isEmpty)
                      ? 'Nama barang wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _jumlah,
              keyboardType: TextInputType.number,
```

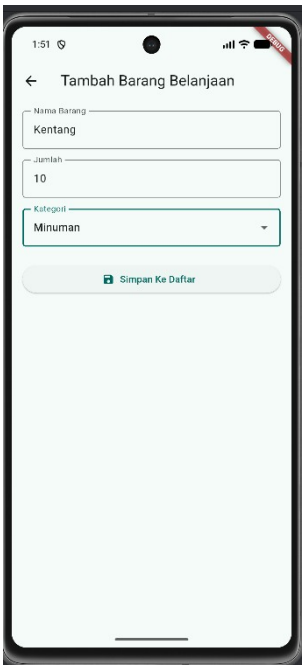
```
              decoration: const InputDecoration(
                labelText: 'Jumlah',
                border: OutlineInputBorder(),
              ),
              validator: (v) {
                if (v == null || v.trim().isEmpty) {
                  return 'Jumlah wajib diisi';
                }
                final n = int.tryParse(v);
                if (n == null || n <= 0) {
                  return 'Jumlah harus angka lebih dari 0';
                }
                return null;
              },
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              decoration: const InputDecoration(
                labelText: 'Kategori',
                border: OutlineInputBorder(),
              ),
              items: _daftarKategori
                  .map((k) => DropdownMenuItem(
                      value: k, child: Text(k)))
                  .toList(),
              onChanged: (v) => setState(() => _kategori = v),
              validator: (v) =>
                  v == null ? 'Pilih kategori' : null,
            ),
            const SizedBox(height: 24),
            ElevatedButton.icon(
              onPressed: _simpan,
              icon: const Icon(Icons.save),
              label: const Text('Simpan Ke Daftar'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Bagian Kode	Penjelasan
class Barang	Model data barang: nama, jumlah, kategori, dan status sudahDibeli.
BelanjaModel extends ChangeNotifier	Satu sumber data bersama untuk halaman daftar dan halaman form.
jumlahBelumDibeli	Getter yang menghitung barang dengan sudahDibeli == false (tanda ! berarti negasi). Nilainya ditampilkan di AppBar.

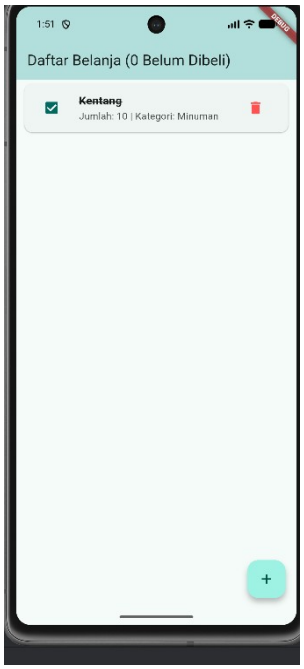
Bagian Kode	Penjelasan
tambah / toggle / hapus	Mengubah data lalu memanggil notifyListeners() agar tampilan ikut diperbarui.
ChangeNotifierProvider di runApp	Membagikan BelanjaModel ke semua halaman.
context.watch<BelanjaModel>()	Dipakai di build() agar AppBar dan daftar otomatis berubah saat data berubah.
context.read<BelanjaModel>()	Dipakai di callback (checkbox, hapus, simpan) karena hanya memanggil method.
model.items.isEmpty ? ... : ...	Menampilkan "Daftar belanja masih kosong" bila belum ada barang.
Card + ListTile	Satu kartu per barang dengan checkbox, nama (dicoret bila sudah dibeli), subjudul jumlah dan kategori, serta ikon hapus merah.
keyboardType: TextInputType.number	Menampilkan keyboard angka untuk kolom jumlah.
int.tryParse(v)	Mengubah teks menjadi angka; menghasilkan null jika bukan angka valid, sehingga aman dari error.
n == null || n <= 0	Validasi jumlah: harus berupa angka dan lebih dari 0.
_daftarKategori.map(...).toList()	Membuat daftar DropdownMenuItem dari list kategori sehingga tidak perlu menulis satu per satu.
ElevatedButton.icon	Tombol "Simpan Ke Daftar" dengan ikon simpan.
int.parse(_jumlah.text)	Mengubah teks jumlah menjadi int; aman dipakai karena sudah lolos validasi.



Gambar 6. Halaman daftar masih kosong



Gambar 7. Form tambah barang



Gambar 8. Daftar setelah barang ditambah dan dicentang

Gambar 6 menampilkan daftar belanja yang masih kosong dengan AppBar "0 Belum Dibeli". Gambar 7 adalah form tambah barang berisi nama, jumlah, dan kategori. Pada Gambar 8 barang "Kentang" muncul di daftar lengkap dengan jumlah dan kategori; setelah dicentang namanya tercoret dan jumlah barang belum dibeli di AppBar menjadi 0.

7. Jawaban Pertanyaan Refleksi

1. Mengapa TextEditingController harus di-dispose?

TextEditingController menyimpan sumber daya dan listener di memori. Jika halaman ditutup tanpa memanggil dispose(), objek tersebut tetap tertahan di memori sehingga terjadi kebocoran memori (memory leak). Dengan memanggil dispose() pada method dispose() milik State, sumber daya dilepas ketika halaman tidak lagi dipakai.

2. Kapan cukup memakai setState, dan kapan sebaiknya beralih ke Provider?

setState cukup untuk state lokal yang hanya dipakai satu widget atau satu halaman, misalnya isi TextField, status centang checkbox, atau pilihan dropdown. Provider sebaiknya dipakai ketika data harus dibagikan ke banyak widget atau halaman, misalnya daftar tugas atau keranjang belanja, karena mengoper data lewat constructor dan callback akan menjadi rumit dan sulit dirawat.

3. Apa yang terjadi bila notifyListeners() lupa dipanggil? Mengapa?

Data di dalam model memang berubah, tetapi tampilan tidak ikut diperbarui. Penyebabnya, notifyListeners() adalah sinyal yang memberi tahu widget-widget yang berlangganan (lewat context.watch atau Consumer) bahwa data telah berubah dan mereka harus dibangun ulang. Tanpa sinyal itu, widget tidak tahu ada perubahan sehingga layar tetap menampilkan data lama.

4. Mengapa di dalam onPressed kita memakai context.read, bukan context.watch?

onPressed adalah callback yang dijalankan saat pengguna menekan tombol, bukan bagian dari proses menggambar UI. Di sana kita hanya perlu mengakses model untuk memanggil method seperti tambah() atau hapus(), tanpa perlu berlangganan perubahan. context.watch hanya boleh dipakai di dalam build() karena ia mendaftarkan widget agar dibangun ulang; memakainya di callback dapat menimbulkan error dan pembangunan ulang yang tidak perlu.

8. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa input pengguna diambil dengan TextField dan TextEditingController, sedangkan form dengan banyak isian divalidasi menggunakan Form, GlobalKey, dan validator pada TextFormField. setState cocok untuk state lokal, namun untuk data yang dipakai banyak halaman diperlukan state management. Dengan ChangeNotifier dan Provider, data disimpan pada satu model bersama; notifyListeners() memberi tahu widget agar diperbarui, context.watch dipakai di build() untuk berlangganan, dan context.read dipakai di callback untuk memanggil method. Konsep ini diterapkan pada aplikasi Daftar Tugas dan Daftar Belanja.